

# DMD Controller Now Playing

Titolo, artista e avanzamento del brano sul pannello, da AirPlay 2, Spotify e Home Assistant  
zedmd-pi 1.12.5 — [github.com/kWGillo/zedmd-pi](https://github.com/kWGillo/zedmd-pi)

## Che cosa fa, in una riga

Il DMD diventa una cassa AirPlay che non suona. Compare fra i dispositivi audio dell’iPhone, accetta il flusso, butta via l’audio e tiene solo i metadati: titolo, artista, album, stato e posizione nel brano. Quello che esce dalle casse vere non cambia di una virgola.

Non è un aggiramento: è il modo previsto. L’alternativa — intercettare il traffico di rete — non funziona e non può funzionare, perché AirPlay 2 cifra il flusso end-to-end con chiavi che nascono dall’accoppiamento. Da un port mirror si vedono i nomi dei dispositivi in mDNS e nient’altro.

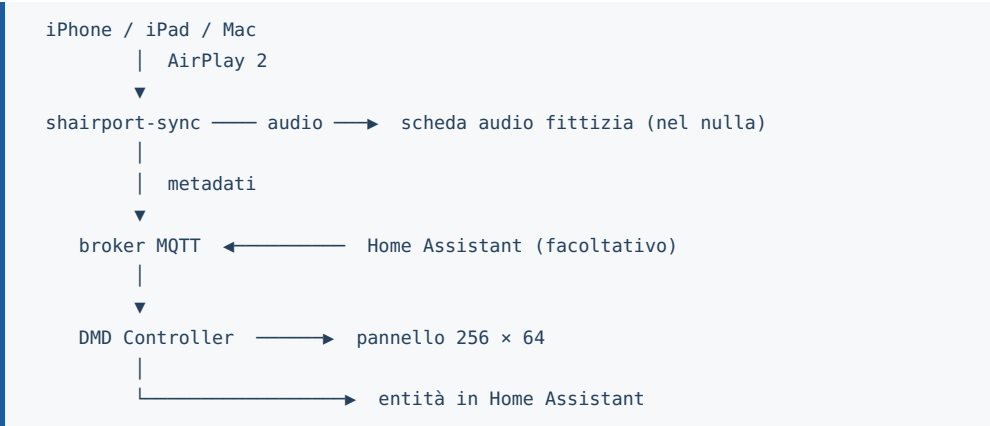
## Che cosa viene rilevato, e che cosa no

| Scenario                                                 | Coperto | Da cosa              |
|----------------------------------------------------------|---------|----------------------|
| iPhone, iPad o Mac che invia AirPlay                     | sì      | shairport-sync       |
| Un gruppo multi-room che include il DMD                  | sì      | shairport-sync       |
| Apple Music, Spotify, Amazon Music, YouTube dal telefono | sì      | shairport-sync       |
| Spotify Connect verso casse vere, computer, Echo         | sì      | API di Spotify       |
| HomePod avviato a voce con Siri                          | no      | serve Home Assistant |
| Amazon Music su un Echo                                  | no      | serve Home Assistant |

La riga che conta è la terza. Al ricevitore AirPlay **non importa quale applicazione stia suonando**: riceve un flusso audio con i suoi metadati, e quelli sono uguali per tutti. Non c’è niente da configurare per Amazon Music o per Apple Music, funzionano perché passano di lì.

Le due righe in fondo sono il confine vero: se la musica nasce e muore su un altro apparecchio senza attraversare il DMD, il DMD non la vede. Per quei casi c’è il **topic esterno**, spiegato al capitolo 6.

## Come sono fatti i pezzi



Il broker MQTT è il punto d'incontro. Il valore predefinito è un Mosquitto installato **sul Raspberry stesso**: così tutto funziona senza Home Assistant. Chi Home Assistant ce l'ha già, scrive l'indirizzo di quel broker e ottiene le due cose insieme, senza installare Mosquitto due volte.

## Il modo veloce

Tutto quello che segue è automatizzato. Lo script viene installato insieme al DMD, quindi da SSH sul Raspberry:

```
sudo /opt/dmd/setup_nowplaying.sh
```

Chiede quattro cose — il nome con cui comparire fra le casse, e dove sta il broker MQTT (lascia vuoto per installarne uno sul Raspberry stesso) — e poi installa Mosquitto, le dipendenze, `nqptp` e `shairport-sync` compilato con AirPlay 2 e i metadati, carica la scheda audio fittizia, scrive `/etc/shairport-sync.conf`, confina i servizi audio ai core 0-2 per lasciare il core 3 al pannello, e compila la sezione MQTT del DMD. In chiusura resta trenta secondi in ascolto del broker: metti musica dal telefono e ti dice se i metadati arrivano davvero.

Sul Pi 4 conta un quarto d'ora, quasi tutto compilazione. Che gira a priorità bassa di CPU e di I/O, di proposito: il pannello ha bisogno di tempi regolari, e una scheda SD sotto sforzo su questo sistema è già stata la causa di guasti veri.

Lo script è **ripetibile**. Ogni passo controlla prima se è già fatto: se `shairport-sync` è già compilato a dovere, salta del tutto la compilazione. Se qualcosa fallisce, si ferma dicendo cosa, e rilanciarlo riprende da lì.

Per controllare lo stato senza modificare niente:

```
sudo /opt/dmd/setup_nowplaying.sh --verifica
```

Se il broker sta sotto Home Assistant, si può anche passare tutto da riga di comando:

```
sudo /opt/dmd/setup_nowplaying.sh --broker 192.168.0.20 --utente dmd --password xxx
```

Restano da fare a mano soltanto le cose che nessuno script può fare al posto tuo: scegliere il DMD fra le casse sul telefono, collegare l'account Spotify (capitolo 6, serve un browser), e le eventuali automazioni di Home Assistant (capitolo 7).

**I capitoli da 1 a 5 descrivono a mano esattamente quello che lo script fa da solo.** Non serve leggerli per installare. Servono se preferisci controllare ogni passo, se il tuo sistema è diverso dal solito, o se qualcosa non funziona e devi capire dove si interrompe la catena.

## 1. Il broker MQTT

Se hai già Mosquitto sotto Home Assistant, salta al capitolo 2 e usa quel suo indirizzo. Altrimenti, sul Raspberry:

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
```

Mosquitto 2.x non accetta connessioni anonime dall'esterno se non glielo si dice. Per un uso in casa, sulla sola rete locale:

```

sudo tee /etc/mosquitto/conf.d/dmd.conf >/dev/null <<'EOF'
listener 1883
allow_anonymous true
EOF

sudo systemctl enable --now mosquitto
systemctl is-active mosquitto

```

`allow_anonymous true` va bene su una rete domestica dietro un router. Se il broker è raggiungibile da fuori, metti un utente e una password ( `mosquitto_passwd` ) e scrivilo nella pagina Musica del DMD.

Prova che risponda:

```

mosquitto_sub -h 127.0.0.1 -t 'prova' &
mosquitto_pub -h 127.0.0.1 -t 'prova' -m 'ciao'

```

## 2. shairport-sync con AirPlay 2 e MQTT

**Prima controlla che cosa hai già.** Il pacchetto della distribuzione spesso è compilato senza AirPlay 2 e senza MQTT:

```
shairport-sync -V
```

Nella riga che stampa devono comparire **AirPlay 2** e **mqtt**. Attenzione alla grafia: la versione attuale scrive `AirPlay2` attaccato, le precedenti `AirPlay-2` col trattino. Se ci sono entrambe le voci vai al capitolo 3, se ne manca anche una sola va ricompilato.

### Dipendenze

```

sudo apt update
sudo apt install -y --no-install-recommends build-essential git autoconf \
    automake libtool pkg-config libpopt-dev libconfig-dev libasound2-dev \
    avahi-daemon libavahi-client-dev libssl-dev libsoxr-dev libplist-dev \
    libplist-utils libsodium-dev libavutil-dev libavcodec-dev libavformat-dev \
    libswresample-dev uuid-dev libgcrypt-dev xxd libmosquitto-dev

# Il file pkg-config di systemd: su Debian recenti e' un pacchetto a parte,
# su quelle precedenti sta in libsystemd-dev. Uno dei due esistera'.
sudo apt install -y systemd-dev || sudo apt install -y libsystemd-dev

```

### nqptp

AirPlay 2 sincronizza gli orologi con PTP e per farlo serve un demone a parte. Senza, shairport-sync in modalità AirPlay 2 non parte.

```

cd ~
git clone https://github.com/mikebrady/nqptp.git
cd nqptp
autoreconf -fi
./configure --with-systemd-startup
make
sudo make install
sudo systemctl enable --now nqptp

```

### shairport-sync

```

cd ~
git clone https://github.com/mikebrady/shairport-sync.git
cd shairport-sync
autoreconf -fi
./configure --sysconfdir=/etc \
            --with-alsa \
            --with-avahi \
            --with-ssl=openssl \
            --with-soxr \
            --with-airplay-2 \
            --with-metadata \
            --with-mqtt-client \
            --with-systemd-startup
make -j4
sudo make install

```

La compilazione sul Pi 4 richiede una decina di minuti.

Prima di lanciare `make`, conviene controllare in due secondi tutto quello che `configure` andrà a cercare, invece di scoprire una dipendenza mancante per volta a compilazione avviata:

```

for m in libplist-2.0 libsodium libavutil libavcodec libavformat \
        libswresample systemd; do
    pkg-config --exists $m || echo "manca: $m"
done
command -v plistutil xxd

```

Tre dettagli che costano tempo se sbagliati:

- Il flag di systemd è `--with-systemd-startup`. `--with-systemd` non esiste più, e autoreconf un flag sconosciuto lo segnala solo come avviso: l'errore vero salta fuori un quarto d'ora dopo, quando manca l'unità di servizio.
- `libplist-utils` serve davvero: `configure` cerca il programma `plistutil`, e per AirPlay 2 senza quello si ferma.
- `systemd-dev` fornisce il file `pkg-config` di systemd, che `configure` interroga per sapere dove mettere l'unità. Su Debian recenti è un pacchetto separato; su quelle precedenti lo stesso file sta in `libsystemd-dev`.

Se `configure` protesta per un'altra libreria, installala e rilancia: l'elenco qui sopra copre i casi normali, ma le versioni cambiano.

### 3. L'audio deve finire nel nulla, ma con un orologio vero

Questa è la parte in cui è facile sbagliare. Servono due cose insieme: scartare i campioni e consumarli al ritmo giusto.

Non usare `/dev/null` né il plugin `null` di ALSA: nessuno dei due limita il ritmo, shairport-sync scriverebbe a velocità infinita e perderebbe il riferimento temporale. In un gruppo multi-room quello è esattamente ciò che fa singhiozzare **tutte** le casse, non solo quella finta.

La soluzione è la scheda audio fittizia del kernel: una scheda ALSA a tutti gli effetti, con timing corretto, che scarta i campioni.

```

sudo modprobe snd_dummy
echo snd_dummy | sudo tee /etc/modules-load.d/snd-dummy.conf
aplay -l | grep -i dummy

```

L'ultimo comando deve elencare una scheda `Dummy`.

## 4. Configurazione di shairport-sync

```
sudo nano /etc/shairport-sync.conf
```

```
general = {
    name = "DMD";                // il nome che vedrai fra le casse
    output_backend = "alsa";
};

alsa = {
    output_device = "hw:CARD=Dummy";
};

mqtt = {
    enabled = "yes";
    hostname = "127.0.0.1";      // o l'indirizzo del broker di Home Assistant
    port = 1883;
    topic = "shairport";        // deve coincidere con la pagina Musica del DMD
    publish_parsed = "yes";      // titolo, artista, album, play/pausa
    publish_raw = "yes";        // serve per prgr, cioè la posizione nel brano
    publish_cover = "no";       // vedi il capitolo 8
};
```

`publish_raw = "yes"` non è facoltativo, e per due motivi. La posizione nel brano viaggia solo sul topic grezzo `prgr`; e soprattutto **la pausa si riconosce solo da `paus`**, anch'esso grezzo. Lo stato leggibile `shairport/playing` resta a `1` per una decina di secondi dopo che hai messo in pausa, finché l'iPhone non chiude la sessione: aspettare quello significherebbe mostrare un brano che avanza mentre è fermo.

Il DMD non si iscrive a `shairport/#` ma al solo livello leggibile più `ssnc/prgr` e `ssnc/paus`. Con `publish_raw` attivo il ramo grezzo porta anche le copertine — centinaia di kilobyte di JPEG per ogni brano — che attraverserebbero il broker per essere poi scartate.

Se il broker vuole le credenziali, aggiungi `username` e `password` dentro il blocco `mqtt`. In quel caso il file non deve restare leggibile da chiunque, ma attenzione a come lo stringi: shairport-sync gira come utente `shairport-sync`, quindi il file va dato al suo gruppo, non lasciato a `root:root`.

```
sudo chgrp shairport-sync /etc/shairport-sync.conf
sudo chmod 640 /etc/shairport-sync.conf
sudo -u shairport-sync test -r /etc/shairport-sync.conf && echo "leggibile"
```

Un file `640 root:root` fa fallire l'avvio con *"Error reading configuration file: file I/O error"*, che non nomina i permessi e manda a cercare altrove.

Poi:

```
sudo systemctl enable --now shairport-sync
systemctl status shairport-sync --no-pager
```

### Verifica prima di toccare il DMD

Metti musica dall'iPhone scegliendo **DMD** fra le casse, e guarda che cosa arriva sul broker:

```
mosquitto_sub -h 127.0.0.1 -t 'shairport/#' -v
```

Devi vedere passare `shairport/title`, `shairport/artist`, `shairport/album` e `shairport/prgr`. Se non arriva niente, il problema è a monte del DMD e va risolto qui.

## 5. Il DMD

Aggiorna alla 1.10 ( `sudo ./update.sh` , oppure il pulsante di aggiornamento nella pagina Impostazioni), poi apri la pagina **Musica**.

**Broker MQTT** — indirizzo, porta, eventuali credenziali. Il topic di shairport-sync deve essere identico a quello scritto in `/etc/shairport-sync.conf` . Salva: la riga di stato deve dire *connesso*.

**Servizi** — accendi *Now Playing* nella pagina Servizi.

Il pulsante **Mostra un brano di prova** mette un brano finto nello stato del player: serve a vedere subito com'è fatto sul pannello, senza dover far partire musica. Sparisce da solo dopo un minuto.

## Dove sta il player fra le priorità

| Priorità  | Sorgente           |
|-----------|--------------------|
| 100       | ZeDMD (Batocera)   |
| 60        | Air Radar          |
| <b>58</b> | <b>Now Playing</b> |
| 55        | Rolling Banner     |
| 50        | Media Player       |
| 10        | Orologio           |

Mentre suona qualcosa il player resta a schermo al posto di foto e banner — è musica continua, non un evento. Lascia passare Air Radar, che dura una decina di secondi. E si toglie di mezzo appena arrivano frame da Batocera: durante una partita comanda il flipper.

Un brano in pausa resta a schermo per il tempo impostato in *Permanenza in pausa* (90 secondi di serie), poi restituisce il display.

## 6. Spotify

Serve **solo** per la musica che non passa da AirPlay. Se ascolti Spotify dall'iPhone mandandolo al DMD come cassa, questa sezione lasciala spenta.

Spotify non accetta più `http://` su un indirizzo di rete: l'indirizzo di ritorno deve essere di loopback. Un DMD headless non ha un browser, quindi si autorizza da un altro computer e si incolla il codice. È il modo previsto per i dispositivi senza schermo.

1. Su `developer.spotify.com/dashboard` crea un'applicazione. Copia il **Client ID**. Il segreto non serve: si usa PKCE.
2. Nella stessa applicazione aggiungi come Redirect URI, scritto identico:  
`http://127.0.0.1:8080/api/spotify/callback`
3. Nella pagina Musica del DMD incolla il Client ID, spunta *Interroga Spotify* e salva.
4. Premi **Genera l'indirizzo di autorizzazione** e apri il collegamento che compare, dal browser di un computer qualsiasi.
5. Autorizza. La pagina finale **non si aprirà**: è previsto, non c'è niente in ascolto su quell'indirizzo. Copia l'intero indirizzo dalla barra del browser.
6. Incollalo nel campo *Indirizzo o codice di ritorno* e premi **Collega l'account**.

I token finiscono in `/var/lib/dmd/spotify.json` , leggibile solo da root, e non compaiono mai in un file di configurazione esportato.

Il DMD chiede a Spotify che cosa sta suonando ogni 8 secondi. Sono circa 450 richieste all'ora, ampiamente entro i limiti.

## 7. Home Assistant

Due direzioni indipendenti.

### Dal DMD verso Home Assistant

Con *Crea le entità automaticamente* attivo, il DMD si presenta da solo tramite MQTT Discovery. In Home Assistant compare un dispositivo con:

- un sensore **Now Playing** — titolo come stato, e artista, album, sorgente, posizione e durata come attributi
- un interruttore per ogni servizio: ZeDMD, Media Player, Rolling Banner, Now Playing, Air Radar, Orologio
- la **luminosità** come numero da 0 a 100

Sono comandabili, non solo leggibili: spegnere l'interruttore in Home Assistant spegne davvero il servizio sul DMD.

Le entità sono legate alla disponibilità: se il servizio si ferma, il testamento MQTT le fa diventare *non disponibili* invece di lasciare valori congelati che sembrano veri.

### Quando Home Assistant riparte

Non serve sorvegliarlo, e il DMD non ha bisogno di sapere dove sia. Ci sono due meccanismi, entrambi automatici:

- le dichiarazioni sono pubblicate con il flag **retain**, quindi restano depositate sul broker e vengono riconsegnate a chiunque si iscriva dopo — compreso un Home Assistant appena riavviato;
- Home Assistant, quando riparte, pubblica [online](#) su [homeassistant/status](#). Il DMD è iscritto a quel topic e si ridichiara. È il modo raccomandato dalla documentazione di Home Assistant.

La risposta al messaggio di nascita è ritardata di un tempo casuale fra mezzo secondo e due secondi e mezzo: al riavvio *tutti* i dispositivi MQTT della casa sentono lo stesso annuncio nello stesso istante, e se rispondessero insieme il broker prenderebbe una raffica.

Nella pagina Musica ci sono comunque due pulsanti manuali:

- **Ridichiara le entità** — scorciatoia, non dovrebbe mai servire.
- **Rimuovi le entità** — cancella il dispositivo da Home Assistant. Serve se cambi l'identificativo o smetti di usare l'integrazione: senza, le vecchie entità resterebbero depositate sul broker come fantasmi.

### Staccare il collegamento

Nella pagina **Servizi**, in alto, c'è un interruttore *MQTT e Home Assistant*: spegne e riaccende il collegamento al broker senza toccare nient'altro. Le impostazioni — indirizzo, utente, password, topic — restano dove sono, quindi riaccendere è un clic e non una ridigitazione.

Sta lì perché è lì che lo si cerca: quando Home Assistant dice cose che non tornano, la prima cosa da fare è staccare e vedere cosa cambia. Prima l'unico modo era questa pagina, in fondo, dentro un modulo di undici campi che vanno risalvati tutti insieme — con il rischio di perdere l'indirizzo del broker mentre si voleva soltanto spegnere.

Spegnendo, il DMD **saluta**: pubblica [offline](#) sul topic di disponibilità, e in Home Assistant le entità diventano *non disponibili* invece di restare congelate sull'ultimo valore. Sul pannello non cambia niente — i servizi continuano per conto loro, e la musica via AirPlay smette di essere annunciata, non di suonare.

### Da Home Assistant verso il DMD

È il modo di coprire quello che il DMD non vede da solo: un HomePod avviato a voce, un Echo, una cassa Sonos. Home Assistant quelle le legge già; basta ripubblicare su un topic.

Un'automazione minima:

```
alias: Now playing sul DMD
trigger:
  - platform: state
    entity_id: media_player.soggiorno
action:
  - service: mqtt.publish
    data:
      topic: dmd/external/nowplaying
      payload: >-
        {{ {
          "title": state_attr('media_player.soggiorno','media_title'),
          "artist": state_attr('media_player.soggiorno','media_artist'),
          "album": state_attr('media_player.soggiorno','media_album_name'),
          "duration": state_attr('media_player.soggiorno','media_duration'),
          "position": state_attr('media_player.soggiorno','media_position'),
          "playing": is_state('media_player.soggiorno','playing')
        } | to_json }}
mode: queued
```

Il DMD accetta anche direttamente i nomi di Home Assistant (`media_title`, `media_artist`, `media_album_name`, `media_duration`, `media_position`, `state`), quindi il template si può accorciare. Un payload vuoto azzerla la sorgente.

Quando più sorgenti hanno qualcosa da dire, comanda AirPlay: se sta arrivando un flusso audio qui, quello è senza dubbio ciò che si sta ascoltando. A parità, vince chi sta suonando su chi è in pausa.

## 8. Perché non c'è la copertina dell'album

È una scelta, non una mancanza. A 64 pixel di lato una copertina è illeggibile, ma soprattutto è fatta quasi solo di **mezzi toni** — ed è esattamente il contenuto che su un pannello S-PWM a refresh basso produce lo sfarfallio. Mettere una miniatura in permanenza sullo schermo significherebbe tenerci il caso peggiore.

Per la stessa ragione il player si disegna in due modi particolari:

**Testo senza antialiasing.** PIL sfuma i bordi delle lettere, e ogni sfumatura è un pixel a intensità intermedia. Il testo qui passa da una maschera ridotta a due soli livelli: acceso o spento. A questa dimensione si legge anche meglio.

**Solo colori pieni.** Con l'opzione *Solo colori pieni* ogni componente viene portata a 0 o 255: restano otto colori, gli stessi di una PNG a palette, quelli che sul pannello non tremolano. La gerarchia fra le righe si ottiene cambiando **tinta** invece che luminosità — bianco per il titolo, ciano per l'artista, blu per l'album, che a parità di saturazione l'occhio legge come più scuro.

Chi vuole colori arbitrari può togliere la spunta, sapendo che cosa comporta.

## 9. Carico sul Raspberry

shairport-sync in modalità AirPlay 2 deve decifrare e decodificare l'AAC anche solo per buttarlo: sono pochi punti percentuali di un core del Pi 4. Il DMD però gira inchiodato al core 3 con priorità realtime, e quel core non va toccato.

```
sudo systemctl edit shairport-sync
```

```
[Service]
AllowedCPUs=0-2
Nice=5
```

Stessa cosa per `nqptp` e `mosquitto` se vuoi essere scrupoloso. Il core 3 resta al pannello.

## 10. Quando qualcosa non va

---

**Il DMD non compare fra le casse dell'iPhone.** È un problema di mDNS: `systemctl status avahi-daemon`. Il Raspberry e il telefono devono stare sulla stessa rete e sulla stessa VLAN.

**Compare ma dà errore appena lo scelgo.** Quasi sempre è nqtp: `systemctl status nqtp`. Senza quel demone AirPlay 2 non si aggancia.

**Suona ma il DMD non mostra niente.** Guarda dove si interrompe la catena:

```
mosquitto_sub -h 127.0.0.1 -t 'shairport/#' -v
```

Se qui non passa nulla, il problema è in shairport-sync (blocco `mqtt` nel file di configurazione, o compilato senza `--with-mqtt-client`). Se passa, il topic scritto nella pagina Musica del DMD non coincide.

**Titolo e artista sì, ma niente barra di avanzamento.** Manca `publish_raw = "yes"`. Alcune applicazioni inoltre non mandano affatto la posizione: in quel caso il player scrive *in riproduzione* al posto della barra, che è corretto.

**Il gruppo multi-room singhiozza da quando c'è il DMD.** Controlla di aver usato `snd_dummy` e non `/dev/null` né il plugin `null` di ALSA. Vedi il capitolo 3.

**Home Assistant non vede il dispositivo.** L'integrazione MQTT di Home Assistant deve puntare allo stesso broker, e il prefisso discovery deve essere quello che usa lei (di serie `homeassistant`). Riavviare Home Assistant è di solito sufficiente, perché i messaggi di discovery sono ritenuti dal broker e alla ripartenza lei li rilegge. Se proprio non compare, il pulsante **Ridichiara le entità** nella pagina Musica forza l'invio; per vedere che cosa arriva davvero:

```
mosquitto_sub -h 127.0.0.1 -t 'homeassistant/#' -v
```

**La libreria MQTT manca.** La pagina Musica lo dice esplicitamente: `sudo apt install python3-paho-mqtt`. Finché manca, Now Playing resta spento e il resto del DMD non se ne accorge.

## 11. Note sulla riservatezza

---

- La **password del broker** viene tolta da ogni configurazione esportata, senza opzione. Un file di configurazione gira: finisce in un backup, in un allegato, in una segnalazione. Dopo un'importazione va riscritta una volta.
- I **token di Spotify** non stanno nella configurazione: vivono in `/var/lib/dmd/spotify.json` con permessi `0600`.
- Il permesso concesso a Spotify è il minimo: leggere lo stato di riproduzione. Nessuna modifica, nessun accesso alla libreria. È revocabile in qualsiasi momento da [spotify.com/account/apps](https://spotify.com/account/apps).
- Il DMD non registra da nessuna parte che cosa hai ascoltato.